Pulsar Search Hands-on Manual

PulsarX + PRESTO Standard Search Pipeline • from filterbank data to candidate diagnostics

Course MPG–CAS Summer School 2026 "Finding New Pulsars", hands-on session | Data test.fil (300 s) | Image ypmen/psrsoft:latest | Mode six standalone commands

0 What This Manual Does

Starting from a 300 s FAST observation (test.fil), we type the commands one by one and run a PulsarX + PRESTO pulsar search pipeline: RFI clean-up, dedispersion, red-noise removal, acceleration search and candidate folding, ending with a candidate diagnostic plot.

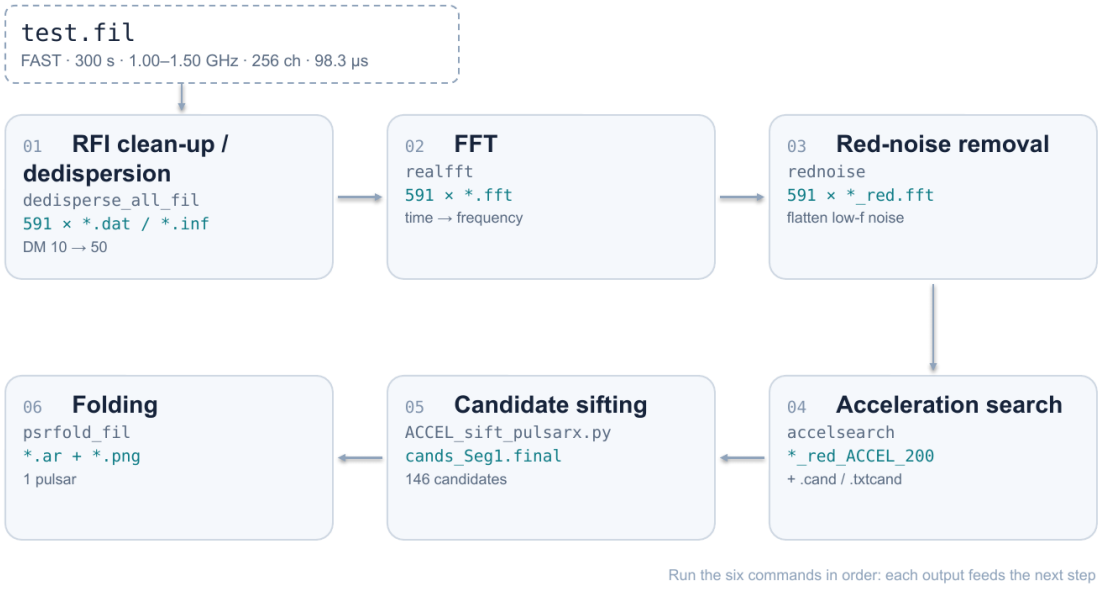


Figure 1 Six stages. Arrows show the order; each output feeds the next step.

1 Import the Image

All software (PulsarX, PRESTO, TEMPO/TEMPO2) is inside the image: no network access and no compilation needed.

The image ypmen/psrsoft:latest is about 8 GB (1.9 GB compressed).

1.1 Load the Image or build the image

```
$ docker load -i psrsoft.tar
# if you received a .tar.gz:
$ gunzip -c psrsoft.tar.gz | docker load
```

or you can also build your image by yourself with the Dockerfile here

https://github.com/ypmen/PulsarX/docs/pulsar_search_hands_on/Dockerfile_psrsoft

```
$ docker build -t psrsoft -f Dockerfile_psrsoft .
```

Check that the image is available:

```
$ docker images | grep psrsoft
ypmen/psrsoft   latest   a9639d9e5281   7.98GB   1.92GB
```

1.2 Self-check: commands available in the container

```
$ docker run --rm ypmen/psrsoft:latest bash -c \
 'which dedisperse_all_fil realfft rednoise accelsearch psrfold_fil psrheader ACCEL_sift_pulsarx.py'
/home/pulsarx/software/bin/dedisperse_all_fil
/home/pulsarx/software/bin/realfft
/home/pulsarx/software/bin/rednoise
/home/pulsarx/software/bin/accelsearch
/home/pulsarx/software/bin/psrfold_fil
/home/pulsarx/software/bin/psrheader
/home/pulsarx/software/bin/ACCEL_sift_pulsarx.py
```

Is Docker installed?

On Linux, docker run hello-world printing a welcome message is enough.
If you get "permission denied", add yourself to the docker group and log in again: sudo usermod -aG docker $USER.

2 Start the Container

```
$ cd ~/pulsar_hands_on
$ docker run -it --name=psrsoft  -e DISPLAY=$DISPLAY   --cap-add=SYS_PTRACE --security-opt
seccomp=unconfined   -v /tmp/.X11-unix:/tmp/.X11-unix   -u root   ypmen/psrsoft bash
```

Inside the container:

```
root@5f3c9a2b7d10:~$ cd pulsar_hands_on
root@5f3c9a2b7d10:~$ ls
ddplan.txt test.fil
```

3 Inspect the Data

```
$ psrheader -f test.fil
Header size (Byte)            : 363
Telescope                     : FAST
Source Name                   : J0000-00
Source RA (J2000)             : 00:00:00.00
Source DEC (J2000)            : 00:00:00.00
Central frequency (MHz)       : 1249.93896484375
Bandwidth (MHz)               : -500
Number of channels            : 256
Sample time (us)              : 98.304000000000002
Number of samples             : 3051520
Observation length (s)        : 299.97662207999997
Number of bits per sample     : 8
```

4 The Six Commands

Stage	cmd
1 dedisperse	dedisperse_all_fil -v -t 4 --td 1 --fd 1 --ddplan ddplan.txt --format presto --rootname J0000-00 -f test.fil
2 realfft	ls *.dat | xargs -n 1 -P 4 realfft
3 rednoise	ls *.fft | xargs -n 1 -P 4 rednoise
4 accelsearch	ls *.fft | xargs -n 1 -P 4 accelsearch -ncpus 1 -zmax 200 -numharm 16
5 sift	ACCEL_sift_pulsarx.py -d ./ -o cands.final
6 fold	psrfold_fil -v -t 4 --td 1 --fd 1 --nsubband 64 --nbin 64 --candfile cands.final --template /home/pulsarx/software/PulsarX/include/template/fast_fold.template --frac 0.0 1.0 --rootname J0000-00 -f test.fil

4.1 Dedispersion (dedisperse_all_fil)

Input .fil, output .dat / .inf

Option	Meaning
-v	print preprocessing / downsampling / RFI stage information
-t 4	4 threads
--td 1 / --fd 1	no extra time or frequency downsampling (the program still downsamples as the ddplan requires)
--ddplan ddplan.txt	the DM plan you wrote
--format presto	write PRESTO .dat + .inf
--rootname J0000-00	output filename prefix (taken from the source name in the header)
-f test.fil	input file

4.2 FFT (realfft)

Input .dat, output .fft

4.3 Red-noise removal (rednoise)

Input .fft, output _red.fft

4.4 Acceleration search (accelsearch)

Option	Meaning
-ncpus 1	number of threads to use
-zmax 200	search range of the frequency derivative: fdot * T^2 = zmax
-numharm 16	sum up to 16 harmonics

Inspect the candidate list:

```
$ head -6 *_DM30.28707254_red_ACCEL_200
 Cand  Sigma  Power  Coherent  Num   Period(ms)   Frequency(Hz)  FFT r    Freq Deriv   FFT z
   1  278.7  38988.  615355.5  16   45.7847(2)    21.8414(1)  6421.031  0.0(1)e-5    0.00
   2  97.08  4832.1   3759.40  16   213.661(5)    4.6803(1)   1375.938  0.0(1)e-5    0.00
```

4.5 Candidate sifting (ACCEL_sift_pulsarx.py)

Input *ACCEL_200, output cands_Seg1.final, columns: #id dm acc F0 F1 F2 S/N

sift does four things:

1. merge candidates from all DMs (reads *_red_ACCEL_*0 and *_red.inf);
2. remove duplicates: a signal seen at several DMs keeps only its best entry;
3. require detection at no fewer than 2 DMs (min_num_DMs = 2), otherwise it is treated as noise;
4. remove harmonically related candidates (P, P/2, 2P ... keep one), then sort by S/N.

Parameter (in script)	Default	Purpose
sifting.sigma_threshold	4.0	discard candidates with S/N below this value
sifting.c_pow_threshold	100.0	coherent-power floor (suppresses single-harmonic noise)
min_num_DMs	2	minimum number of DMs a candidate must appear in
sifting.long_period	15.0 s	longest period; longer is treated as red noise / instrumental effect
sifting.r_err	1.1 bin	frequency tolerance for calling two hits the same candidate

⚠ If sift fails with ValueError: Cannot find a '_DM<float>' DM in the candidate filename ...
Cause: PRESTO's sifting.py in this image only accepts 2-3 decimals in the DM filename, while PulsarX writes 8 decimals, so relaxing DM_re to _DM(\d+\.\d+) is required (already patched in the image).

4.6 Folding (psrfold_fil)

Option	Meaning
--candfile	candidate list; folded row by row, producing one .ar and one .png per candidate
--template	standard folding template (the software infers the parameters from the input data)
--nsubband 64	64 sub-bands in frequency, used for the frequency-phase plot that checks the DM
--nbin 64	64 phase bins
--frac 0.0 1.0	use 0-100% of the data (to fold a single segment of a segmented search)
--rootname	output prefix

Input .fil, output one pair per candidate; the index is the row number in the candidate list:

```
J0000-00_Seg1_60515.0066875306_cfbf00000_00001.ar    # PSRCHIVE archive (open with psrplot/psrstat)
J0000-00_Seg1_60515.0066875306_cfbf00000_00001.png   # diagnostic plot, shown below
J0000-00_Seg1_60515.0066875306_cfbf00000.cands       # parameters refined by folding
```

Folding Diagnostic Plot

Copy the file out to view

```
$ docker cp psrsoft:/home/pulsarx/pulsar_hands_on/J0000-00_Seg1_60515.*_cfbf00000_00001.png ./
```

